

Introducción a la Programación I

Clase 5: Listas

¿Qué es una lista?

Una **lista** es una colección **ordenada** y **mutable** de elementos.

```
# Lista vacía
mi_lista = []

# Lista con elementos
frutas = ["manzana", "banana", "naranja"]
numeros = [1, 2, 3, 4, 5]
mixta = ["hola", 42, 3.14, True]
```

- Se escribe con **corchetes** `[]`
- Los elementos se separan con **comas**
- Puede contener cualquier tipo de dato (¡incluso mezclarlos!)
- Es **mutable**: podemos cambiarla después de crearla

Índices

Python numera los elementos de una lista empezando desde **0**:

```
frutas = ["manzana", "banana", "naranja"]  
#           0           1           2
```

También se pueden usar **índices negativos** para contar desde el final:

```
frutas = ["manzana", "banana", "naranja"]  
#           -3           -2           -1
```

Índice positivo	Índice negativo	Elemento
0	-3	"manzana"
1	-2	"banana"
2	-1	"naranja"

Acceder a elementos

Usamos el índice entre corchetes para acceder a un elemento:

```
frutas = ["manzana", "banana", "naranja"]

print(frutas[0])    # manzana
print(frutas[1])    # banana
print(frutas[2])    # naranja

# Índices negativos
print(frutas[-1])   # naranja (último)
print(frutas[-2])   # banana (penúltimo)
```

El elemento devuelto es un valor normal que podés usar en cualquier expresión:

```
print(f"Mi fruta favorita es la {frutas[0].upper()}")
# Mi fruta favorita es la MANZANA.
```

len() con listas

La función `len()` devuelve la **cantidad de elementos** de una lista:

```
frutas = ["manzana", "banana", "naranja"]  
print(len(frutas))    # 3  
  
vacía = []  
print(len(vacía))     # 0  
  
numeros = [10, 20, 30, 40, 50]  
print(len(numeros))   # 5
```

Útil para obtener el **último índice válido**:

```
ultimo_indice = len(frutas) - 1  
print(frutas[ultimo_indice])    # naranja  
# equivalente a frutas[-1]
```

Modificar elementos

Las listas son **mutables**: podemos cambiar sus elementos:

```
frutas = ["manzana", "banana", "naranja"]  
print(frutas)    # ['manzana', 'banana', 'naranja']  
  
frutas[1] = "pera"  
print(frutas)    # ['manzana', 'pera', 'naranja']  
  
frutas[0] = "uva"  
print(frutas)    # ['uva', 'pera', 'naranja']
```

La sintaxis es simple:

```
lista[indice] = nuevo_valor
```

Agregar elementos: append()

`append()` agrega un elemento **al final** de la lista:

```
frutas = ["manzana", "banana"]
frutas.append("naranja")
frutas.append("uva")

print(frutas)    # ['manzana', 'banana', 'naranja', 'uva']
```

Es muy útil para construir listas dinámicamente:

```
numeros_pares = []
for i in range(0, 10, 2):
    numeros_pares.append(i)

print(numeros_pares)    # [0, 2, 4, 6, 8]
```

Agregar elementos: insert()

`insert()` agrega un elemento en **cualquier posición**:

```
frutas = ["manzana", "banana", "naranja"]

frutas.insert(0, "uva")          # inserta al inicio
print(frutas)                   # ['uva', 'manzana', 'banana', 'naranja']

frutas.insert(2, "pera")         # inserta en posición 2
print(frutas)                   # ['uva', 'manzana', 'pera', 'banana', 'naranja']
```

La sintaxis es:

```
lista.insert(indice, valor)
```

Los elementos que estaban en esa posición y después se **desplazan** una posición.

Eliminar elementos: del

`del` elimina un elemento por su **índice**:

```
frutas = ["manzana", "banana", "naranja", "uva"]

del frutas[1]
print(frutas)    # ['manzana', 'naranja', 'uva']

del frutas[0]
print(frutas)    # ['naranja', 'uva']
```

⚠ Una vez eliminado con `del`, el elemento ya no está disponible.

```
# También podés borrar toda la lista:
del frutas
# print(frutas)    # NameError: la variable ya no existe
```

Eliminar elementos: slice vacío

Podés reemplazar una porción de la lista con una lista vacía para eliminar elementos:

```
numeros = [1, 2, 3, 4, 5, 6]

numeros[2:4] = []      # elimina los elementos en posiciones 2 y 3
print(numeros)         # [1, 2, 5, 6]

numeros[1:3] = []      # elimina posiciones 1 y 2
print(numeros)         # [1, 6]
```

Esta técnica es útil cuando querés eliminar un rango de elementos de una vez.

Slicing de listas

El **slicing** permite obtener una porción de la lista:

```
letras = ["a", "b", "c", "d", "e"]  
  
print(letras[1:4])    # ['b', 'c', 'd'] → desde 1, sin incluir 4  
print(letras[2:])     # ['c', 'd', 'e'] → desde 2 hasta el final  
print(letras[:3])     # ['a', 'b', 'c'] → desde el inicio hasta 3 (sin incluir)  
print(letras[:])      # ['a', 'b', 'c', 'd', 'e'] → copia completa
```

La sintaxis es:

```
lista[inicio:fin]    # fin no está incluido
```

Slicing: más ejemplos

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Con step (tercer parámetro)
print(numeros[::2])      # [0, 2, 4, 6, 8] → de a 2
print(numeros[1::2])     # [1, 3, 5, 7, 9] → impares
print(numeros[::-1])     # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] → invertida

# Combinando
print(numeros[2:8:2])    # [2, 4, 6]
```

El `[:]` hace una **copia** de la lista (no una referencia):

```
original = [1, 2, 3]
copia = original[:]
copia[0] = 99
print(original)    # [1, 2, 3] ← no cambió
print(copia)       # [99, 2, 3]
```

Strings como listas

Un **String** es, en muchos sentidos, una lista de caracteres:

```
nombre = "Python"

print(len(nombre))      # 6
print(nombre[0])        # P
print(nombre[1])        # y
print(nombre[-1])       # n

# Slicing también funciona
print(nombre[0:3])      # Pyt
print(nombre[::-1])     # nohtyP (invertido)
```

Todo lo que aprendimos sobre índices y slicing aplica también a los strings.

Strings vs Listas: diferencia clave

La gran diferencia: los strings son **inmutables**.

```
frutas = ["manzana", "banana"]
frutas[0] = "uva"           #  OK, las listas son mutables
print(frutas)               # ['uva', 'banana']

nombre = "Python"
nombre[0] = "J"             #  TypeError: 'str' object does not support item assignment
```

Para "modificar" un string, hay que crear uno nuevo:

```
nombre = "Python"
nuevo_nombre = "J" + nombre[1:]
print(nuevo_nombre)        # Jython
```

Recorrer strings con índices

Como los strings son indexables, podemos recorrerlos carácter por carácter:

```
palabra = "hola"

for i in range(len(palabra)):
    print(f"Posición {i}: {palabra[i]}")
# Posición 0: h
# Posición 1: o
# Posición 2: l
# Posición 3: a
```

O más directamente:

```
for letra in palabra:
    print(letra)
```

Listas de listas

Una lista puede contener otras listas (**listas multidimensionales**):

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Para acceder a los elementos usamos **doble índice** [fila][columna] :

```
print(matriz[0])      # [1, 2, 3] → primera fila  
print(matriz[0][0])   # 1       → fila 0, columna 0  
print(matriz[1][2])   # 6       → fila 1, columna 2  
print(matriz[2][1])   # 8       → fila 2, columna 1
```


Ejemplo: tabla de calificaciones

```
calificaciones = [  
    ["Ana",    [9, 8, 10, 7]],  
    ["Carlos", [6, 7, 8, 9]],  
    ["María",  [10, 10, 9, 8]]  
]  
  
for alumno in calificaciones:  
    nombre = alumno[0]  
    notas = alumno[1]  
    promedio = sum(notas) / len(notas)  
    print(f"{nombre}: promedio {promedio:.1f}")
```

Salida:

```
Ana: promedio 8.5  
Carlos: promedio 7.5  
María: promedio 9.2
```

Resumen

Operación	Sintaxis	Descripción
Crear lista	<code>lista = [a, b, c]</code>	Lista con elementos
Acceder	<code>lista[i]</code>	Elemento en posición i
Longitud	<code>len(lista)</code>	Cantidad de elementos
Modificar	<code>lista[i] = x</code>	Cambia elemento en posición i
Agregar al final	<code>lista.append(x)</code>	Agrega x al final
Insertar	<code>lista.insert(i, x)</code>	Inserta x en posición i
Eliminar	<code>del lista[i]</code>	Elimina elemento en posición i
Slice	<code>lista[a:b]</code>	Porción desde a hasta b (sin incluir)
Copia	<code>lista[:]</code>	Copia completa de la lista